TECNICATURA UNIVERSITARIA EN DESARROLLO WEB · FCAD · UNER

# Unidad 7
# Publicar y actualizar

Desarrollo para Móviles · 2026 — Segundo cuatrimestre

Prof. Titular: Lic. Franco Ruis Dias · JTP: Lic. Germán Cueva

Expo SDK 57 · React Native 0.86 · React 19.2

UNIDAD 7

# Qué vamos a ver

La clase anterior terminó con un binario firmado. Falta que llegue a alguien.

## Temas

- Preparar la app: ícono, splash, ficha de tienda
- Cuentas de desarrollador y sus requisitos
- **EAS Submit** y los canales de prueba
- **EAS Update**: actualizaciones por aire
- Después de publicar: errores, uso y rendimiento

## Objetivos

- Poder subir una app a una tienda de punta a punta
- Saber qué se corrige sin volver a publicar y qué no
- Enterarse de que la app se rompió sin que te lo cuenten
- Dejar el TP integrador listo para entregar

UNIDAD 7 · PARTE 1

# Preparar la app

Lo que la tienda mira antes de mirar el código.

# El ícono

```
{
  "icon": "./assets/images/icon.png",
  "android": {
    "adaptiveIcon": {
      "foregroundImage": "./assets/images/icon-frente.png",
      "monochromeImage": "./assets/images/icon-mono.png",
      "backgroundColor": "#0B3A5D"
    }
  }
}
```

**1024×1024, PNG, cuadrado exacto, sin transparencias ni esquinas redondeadas.** El sistema operativo recorta la forma que corresponda; EAS Build genera los demás tamaños.

# La pantalla de presentación

```
{
  "plugins": [
    ["expo-splash-screen", {
      "image": "./assets/images/splash-icon.png",
      "backgroundColor": "#FFFFFF",
      "imageWidth": 200,
      "dark": {
        "image": "./assets/images/splash-icon-dark.png",
        "backgroundColor": "#000000"
      }
    }]
  ]
}
```

**No se prueba en Expo Go ni en un development build.** Expo Go muestra el ícono de la app, y el development build trae su propio splash. Hay que verlo en un build de `preview` o de `production`.

# El app.json de producción

| Campo | Qué es |
|---|---|
| `name` | El nombre debajo del ícono, en el teléfono. |
| `android.package` `ios.bundleIdentifier` | La identidad de la app: `ar.edu.uner.miapp`. **No se puede cambiar nunca más** una vez publicada. |
| `version` | La versión visible en la tienda. |
| permisos y textos de uso | Los de la Unidad 4: cámara, ubicación. iOS exige explicar para qué. |

> Elegir mal el identificador es el error irreversible de la unidad. Se decide una vez, con calma.

# Android 16 o no hay envío

### La fecha

Desde el **31 de agosto de 2026**, Google Play rechaza apps nuevas y actualizaciones que apunten a menos de **Android 16 (API 36)**.

### Dónde estamos

Expo apunta a API 36 desde el **SDK 54**. Con SDK 57 el template por defecto ya cumple: no hay que tocar nada.

> Es la misma decisión que trajo el **edge-to-edge obligatorio** que vimos en la Unidad 3. Apuntar a una versión nueva de Android no es solo un número: cambia cómo se comporta la app.

# La ficha de la tienda

Es trabajo aparte del código, y suele subestimarse.

### Textos

Nombre, descripción corta y larga, palabras clave. Es lo que decide si alguien la encuentra.

### Imágenes

Capturas por tamaño de pantalla, gráfico de cabecera, ícono de alta resolución.

### Declaraciones

Política de privacidad (URL obligatoria), clasificación de contenido, qué datos recolecta la app.

> La política de privacidad es un requisito de las dos tiendas, aunque la app no recolecte nada. Hay que tenerla publicada en una URL accesible.

UNIDAD 7 · PARTE 2

# Las cuentas y el envío

Qué cuesta cada tienda, qué exige, y cómo se sube el binario.

# Las dos cuentas

### Google Play

**25 USD**

pago único, de por vida

Revisión más rápida. Varios canales de prueba.

### Apple App Store

**99 USD**

por año, mientras publiques

Revisión humana más estricta. TestFlight incluido.

Sin cuenta se puede **construir y firmar** para Android, y repartir por distribución interna. Lo que no se puede es estar en la tienda.

# La barrera que nadie espera

### 12 testers · 14 días

Una cuenta **personal** de Google Play creada después de noviembre de 2023 tiene que correr una prueba cerrada con al menos **12 personas distintas**, aceptadas y activas durante **14 días seguidos**, antes de poder pedir acceso a producción.

### Detalles que importan

- Doce cuentas de Google reales, entradas por el link de invitación.
- Emuladores y cuentas duplicadas no cuentan.
- Si alguien se sale, el reloj se reinicia.
- Las cuentas de **organización** están exentas.

Traducción para el TP: **publicar en Play Store no entra en un cuatrimestre.** Por eso entregamos por distribución interna.

# EAS Submit en Android

```
# Sube el último build de producción a Play Console
eas submit --platform android

# O todo junto: compilar y enviar
eas build --platform android --auto-submit
```

Antes hay que hacer dos cosas una sola vez: crear la app en Play Console, y darle a EAS una **clave de cuenta de servicio** de Google para que suba en tu nombre.

> El primer envío entra al canal de **prueba interna** y la app queda en borrador hasta que completes la ficha.

# EAS Submit en iOS

```
{
  "submit": {
    "production": {
      "ios": { "ascAppId": "1234567890" }
    }
  }
}
```

```
eas submit --platform ios
```

El `ascAppId` es el **Apple ID** numérico que App Store Connect le asigna a la app. Aparece en App Information > General Information.

> Todo lo que se sube a la App Store pasa primero por **TestFlight**. No hay forma de saltearlo.

# Los canales de prueba

### Google Play

- **Interna** — hasta 100 personas, sin revisión, disponible en minutos.
- **Cerrada** — por lista o por grupo. Es la de los 12 testers.
- **Abierta** — cualquiera puede sumarse desde la ficha.
- **Producción** — todo el mundo.

### Apple

- **TestFlight interno** — hasta 100 miembros del equipo, sin revisión.
- **TestFlight externo** — hasta 10.000 personas, con una revisión previa más liviana.
- **App Store** — revisión completa.

> Los canales de prueba permiten lanzar por etapas: probás con pocos, mirás cómo anda, y recién después promovés la misma build a producción.

UNIDAD 7 · PARTE 3

# Actualizar sin pasar por la tienda

EAS Update y el límite entre lo nativo y lo que se puede cambiar.

# El problema

Viernes a la noche. La app está publicada. Un usuario reporta que el botón de pagar tira error.

### Sin actualizaciones por aire

Corregís una línea, compilás, subís, esperás la revisión, y aun así el usuario tiene que ir a la tienda a actualizar.
Días, en el mejor caso.

### Con EAS Update

Corregís la línea, publicás la actualización y la app la toma en el próximo arranque.
Minutos.

# Dos capas

Una app compilada tiene una parte que no se puede tocar y otra que sí.

### Capa nativa — fija

El código compilado, las librerías nativas, los permisos declarados, la versión del SDK.

Cambiarla exige un build nuevo y pasar por la tienda.

### Capa de JavaScript — intercambiable

Tus componentes, tus pantallas, los estilos, las imágenes del bundle.

Esto es lo que reemplaza una actualización.

> EAS Update reemplaza la capa de arriba dejando la de abajo intacta. Todo el mecanismo se reduce a garantizar que las dos sean compatibles.

# Publicar una actualización

```
# Una vez por proyecto
npx expo install expo-updates
eas update:configure

# Después de esto hace falta UN build nuevo, que ya incluye la librería

# Cada vez que quieras publicar
eas update --channel production --message "Arreglo del botón de pagar"
```

> El **canal** conecta builds con actualizaciones: un build compilado para `production` solo recibe lo publicado en ese canal. Así se prueba una actualización en `preview` sin tocar a los usuarios.

# runtimeVersion, el seguro

Es la propiedad que garantiza que una actualización solo llegue a builds cuya capa nativa la soporta.

### `"1.0.0"`

Manual. Vos la subís cada vez que tocás algo nativo. Control total y olvidos posibles.

### Política `appVersion`

Sigue a `version`. Simple, pero si te olvidás de subir la versión, quedan desincronizadas.

### Política `fingerprint`

Calcula una huella de todo lo que afecta la capa nativa. Cambia sola cuando hace falta.

> Si publicás una actualización que usa una librería nativa que el build no tiene, la app **se cae al arrancar**. `expo-updates` intenta volver a la versión anterior, pero es un incidente real.

# Qué se puede y qué no

## SÍ, POR AIRE

- Corregir un bug de JavaScript
- Cambiar textos, traducciones, estilos
- Reordenar o rediseñar pantallas
- Cambiar imágenes del proyecto

## NO: HACE FALTA BUILD NUEVO

- Agregar o actualizar una librería nativa
- Pedir un permiso nuevo
- Subir de versión de Expo SDK
- Cambiar ícono, splash o nombre de la app

Y sigue valiendo la regla de fondo: **las actualizaciones tienen que respetar las reglas de las tiendas.** Cambiar el comportamiento de la app por aire para esquivar la revisión es motivo de baja.

# Cuando la actualización sale mal

### Lanzamiento gradual

Publicás a un porcentaje de usuarios, mirás los errores en el panel, y recién después al resto.

### Republicar

Volvés a publicar una actualización anterior que funcionaba, encima de la que falló. Como un commit nuevo, no un borrado.

### Volver al binario

Un rollback devuelve a los usuarios a la versión que venía adentro del build instalado.

> Desde el SDK 55 las actualizaciones viajan como **diferencias** del bytecode de Hermes, no como el bundle entero: la descarga baja alrededor de un 75 %.

UNIDAD 7 · PARTE 4

# Después de publicar

Ahora la app corre en teléfonos que no podés ver.

# Las tres preguntas

En desarrollo tenías la consola, el depurador y el teléfono en la mano. En producción no tenés nada de eso: tenés que **instrumentar la app para que te cuente**.

### ¿Se rompe?

Errores y cierres inesperados, con el stack trace y el contexto del dispositivo.

Crash reporting — Sentry

### ¿La usan?

Cuánta gente la abre, con qué versión, en qué plataforma, y qué pantallas recorre.

Analítica — EAS Insights, PostHog

### ¿Anda rápido?

Cuánto tarda en arrancar y en responder, en teléfonos reales y redes reales.

Rendimiento — EAS Observe

> Sin esto, la única fuente de información sobre una app publicada son las **reseñas de una estrella**. Llegan tarde, sin stack trace y por escrito.

# Sentry: enterarte de que se rompió

```
npx @sentry/wizard@latest -i reactNative
```

Instala `@sentry/react-native`, agrega su plugin de Metro y su config plugin al `app.json`. En el código, una línea al arrancar:

```
import * as Sentry from '@sentry/react-native';

Sentry.init({ dsn: process.env.EXPO_PUBLIC_SENTRY_DSN });

export default Sentry.wrap(RootLayout);
```

> **Los source maps son el punto entero.** Un build de release está minificado: sin ellos el error llega como `a.b is not a function (index.bundle:1:84213)`. EAS Build los sube solo; después de un `eas update` hay que subirlos a mano.

UNIDAD 7 · PARTE 4 · DESPUÉS DE PUBLICAR

# Analítica y rendimiento

### Lo que ya tenés en EAS

**EAS Insights** muestra uso por plataforma, por versión de tienda y en el tiempo. Sale de los pedidos de EAS Update; agregando `expo-insights` se vuelve preciso.

**EAS Observe** mide el arranque en frío, el tiempo hasta el primer dibujo y hasta que la app responde, en sesiones reales.

### Si necesitás más

**PostHog** — eventos de producto, feature flags y repetición de sesión; se integra desde EAS CLI y etiqueta los eventos por release.

**LogRocket**, **Vexo**, **BugSnag** — alternativas con documentación propia en Expo.

> Todo esto se cruza con lo de la parte anterior: los eventos y los errores quedan **etiquetados por canal y por ID de actualización**. Eso es lo que te deja responder «¿esto empezó con el update de ayer?» — y decidir si hay que revertirlo.

UNIDAD 7 · PARTE 5

# La entrega del TP

Cómo se aplica todo esto al trabajo integrador.

# Cómo se entrega el integrador

```
# Perfil preview: APK de release, instalable, sin cuenta de tienda
eas build --platform android --profile preview
```

## Qué se entrega

- El **link del build** que devuelve EAS.
- El repositorio, con el `eas.json` incluido.
- Ícono y splash propios ya configurados.

## Qué tiene que poder hacer la cátedra

- Abrir el link desde un Android e instalar.
- Usar la app **sin** Metro ni la computadora de nadie.
- Ver el nombre y el ícono del grupo en el teléfono.

> No se pide publicar en Play Store: por la regla de los 12 testers, no entra en el cuatrimestre.

UNIDAD 7

# Documentación

- docs.expo.dev/build — EAS Build y `eas.json`
- docs.expo.dev/submit — EAS Submit
- docs.expo.dev/eas-update — actualizaciones y runtimeVersion
- docs.expo.dev/distribution — buenas prácticas de tienda
- docs.expo.dev/monitoring/services — Sentry, Insights, Observe y el resto